ETH zürich

D-INFK Lehre

RL in LLMs

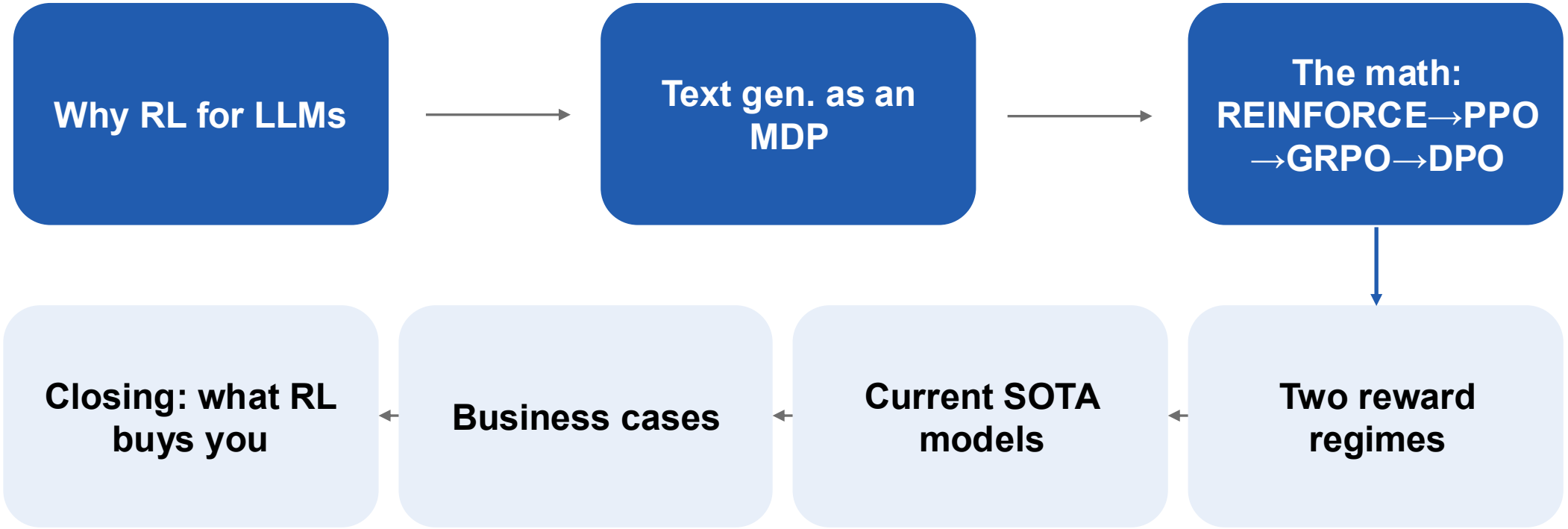
Weekend 4: Reinforcement Learning

Piyushi Goyal

Roadmap of the Weekend

1. **Re-intro to RL**
   Agents, environments, rewards — the vocabulary everything else builds on
2. **Policy Gradients**
   REINFORCE: turning “try it and see” into a gradient
3. **Actor-Critic**
   A learned baseline that cuts variance
4. **GAE & PPO**
   The advantage estimator and the clip that ships in production
5. **RL in LLMs** ← Currently
   Same machinery, new environment: the page the model is writing

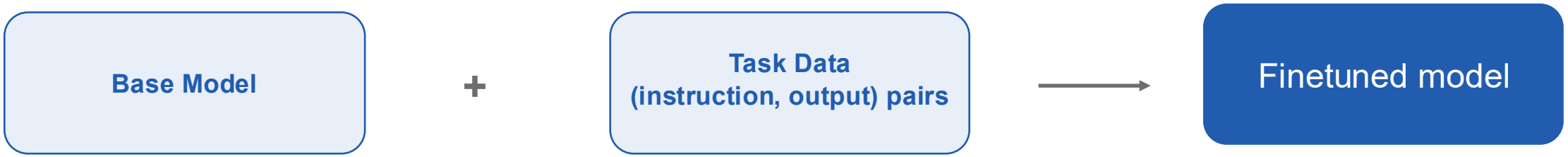
Today's Arc


Why RL for LLMs
Text gen. as an MDP
The math: REINFORCE→PPO →GRPO→DPO
Two reward regimes
Current SOTA models
Business cases
Closing: what RL buys you

Part 1: Why RL for LLMs

Maximum Likelihood Trains Imitation

- The standard fine-tuning recipe:
  - Given input–output pairs, maximize the likelihood of the next token given the previous ones; also, known as Maximum Likelihood Estimation (MLE)
- Intuition: copy behaviors in the dataset, and hope it generalizes



MLE: maximize the likelihood of the next token given the previous ones

This “hope” fails in three specific, nameable ways.

Imitation Fails and RL fixes it

Failure	What goes wrong	RL changes
Task mismatch	The objective (likelihood) isn't the goal. Being helpful, non-offensive etc. are not differentiable through argmax decoding.	The task criterion is directly optimized via the reward
Data mismatch	No data for everything we want; Some of it is bad; MLE can't say “this is worse than that.”	Data is generated by the model; the reward says how to use it
Exposure bias	Trained on ground-truth prefixes, tested on its own; Errors compound and are never corrected. *Shows up on chat as drift, repetition loops, and degeneration.*	Model generations are in the training loop. Test time resembles train time

What Else Could You Do?

Cheaper alternatives that use the same reward function:

Best-of-n sampling: generate N completions, keep the highest-reward one at inference time. No training at all: the reward only ever picks; it never updates a single weight.

Rejection-sampling fine-tuning (STaR): generate, filter by the same reward, then fine-tune on the survivors with plain MLE. One round of RL's data loop, none of its optimizer machinery.

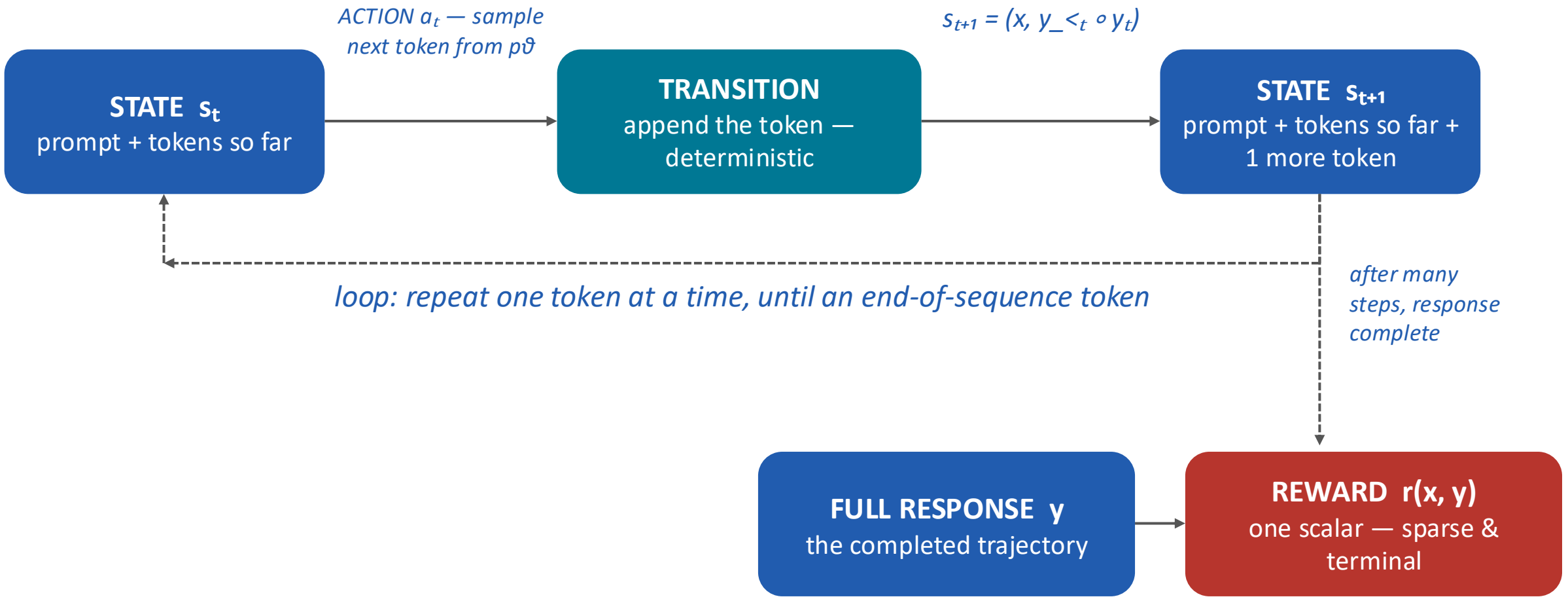
Same reward, less machinery: both alternatives spend the exact same reward signal RL will use later. The question is never “do you have a reward,” it's “how much machinery do you want to pay for using it.”

Part 2: Text Generation as an MDP

The Mapping

RL	Robot Arm	LLM
Agent	The robot arm	The language model
Policy	$\pi(a \mid s)$: which motor command, given what it senses	$p\theta(y_t \mid y_{<t}, x)$: which token, given the text so far
State	Joint angles, gripper position, camera image	The prompt + tokens generated so far
Action	Move a joint by a small amount	Generate the next token
Transition	Physics: noisy, never quite repeatable	Append the token: deterministic
Reward	Was the cup placed on the shelf	Is the final answer correct / is the response good

The Full Loop



What Makes This MDP Unusual

- **Action space ≈ 10^5:** the entire vocabulary, at every step
- **Transitions are deterministic:** all randomness is the policy's own sampling
- **Reward is sparse and terminal:** one scalar at the end of a 1,000-token trajectory
- **Initialization is not random:** the policy starts as an already-strong pretrained model

Part 3: The Math

REINFORCE: the Score-Function Estimator

- The cross-entropy **gradient on the model's OWN sample**, scaled by the reward. Good sample → step toward it. Bad sample → step away.
- RL-for-LLMs is “weighted cross-entropy on your own samples,” **not a wholly new kind of training.**

$$\hat{g} = r(x, \hat{y}) \sum_t \nabla_\theta \log p_\theta(\hat{y}_t \mid \hat{y}_{<t}, x)$$

Subtract a Baseline

- **The problem:** when reward is 0 for most samples, those samples contribute nothing to the gradient (r multiplies the whole term). We only ever learn from the rare successes, and never get an explicit “don't do this” signal.
- **The fix:** subtract a baseline b(x), the score you'd expect on this prompt. What now multiplies the log-prob is an advantage: how much better than expected was this answer?
- b(x) is learned by a value network; a second model, trained alongside the policy, whose only job is to predict the expected score.

$$\hat{g} = \boxed{(r(x, \hat{y}) - b(x))} \nabla_\theta \log p_\theta(\hat{y} \mid x)$$

Prompt	Reward	Baseline b(x)	Advantage A
“Summarize this paper: …”	0.8	0.75	+0.05
“Summarize this paper: …”	0.3	0.75	−0.45
“Prove this theorem: …”	0.3	0.10	+0.20

That second network is expensive to train. But because we're dealing with language models, there's a much simpler way to get b(x).

Can you guess?

GRPO: Group Relative Policy Optimization

- **The idea:** don't learn b(x) at all. Sample K answers to the same prompt and use the group's own average score as the baseline. **Better than the group average → reinforced; worse → discouraged**.
- **What changes vs PPO:** the value network disappears. For a language model, generating a few more answers is cheap; training an accurate second network is not.
- Example below: reversing the word “hello”.

Sample (K=4)	olhl	ollh	ollh	olleh (correct)
Reward r	0	0	0	1
r − mean	−0.25	−0.25	−0.25	+0.75
Advantage A (÷ std)	−0.577	−0.577	−0.577	+1.732

$$A^{(i,k)} = \frac{r^{(i,k)} - \text{mean}\left(r^{(i,1)}, \ldots, r^{(i,K)}\right)}{\text{std}\left(r^{(i,1)}, \ldots, r^{(i,K)}\right)}$$

K is typically 4–16. A larger K gives a more reliable baseline, at the cost of more generations per prompt. This is the optimizer behind DeepSeek-R1 and most reasoning models trained since.

DPO: Direct Preference Optimization

- **The idea:** when all you have is pairs of answers where a human said “this one is better”, you can skip the RL loop. The best policy can be written down in closed form, so it collapses into one supervised loss.
- **What it removes:** no reward model, no value network, no generation during training. Just the policy and a frozen copy of it.
- **Where you see it:** the default alignment step for most open-weight chat models (Llama, Mistral, Qwen), and behind the preference fine-tuning APIs at OpenAI and Azure.
- **The trade-off:** it is off-policy, trained once on a fixed set of pairs, so it can never correct the mistakes the model is making right now.

$$\mathcal{L}_{\mathrm{DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{p_\theta(y_+ \mid x)}{p_0(y_+ \mid x)} - \beta \log \frac{p_\theta(y_- \mid x)}{p_0(y_- \mid x)}\right)\right]$$

Part 4: Two Reward Regimes

RLVR vs RLHF

	RLVR (Reinforcement-learning with Verifiable Rewards)	RLHF (Reinforcement learning through Human Feedback)
Reward	A function you can write	A model you must learn
Example	Exact-match, unit tests, checkable answer	Bradley–Terry preference model
Works where	A checker exists (math, code, proofs)	“Good” is fuzzy (tone, helpfulness)
Failure mode	Verifier / parser gaming	Length bias, helpfulness, over-optimization

DeepSeek-R1: the RLVR Headline Result

- Task: math problems with a checkable final answer. No SFT cold start.
- Reward: 0/1 answer check + a format reward.
- **Result: long chains of thought, self-verification, and backtracking emerged from outcome-only reward.**
- Nobody wrote “double-check yourself if you're unsure” into the reward function. That behaviour was simply cheaper for the optimizer to discover than not to.
- Ties back to the “Exposure Bias” mentioned in Part 1: the model is finally training on its own error states, so it can learn to recover from them.
- Fun fact: famously known as the ‘aha’ moment!

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a-\sqrt{a+x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a-\sqrt{a+x}} = x$, let's start by squaring both $\cdots$

$\left(\sqrt{a-\sqrt{a+x}}\right)^2 = x^2 \implies a-\sqrt{a+x} = x^2$.

Rearrange to isolate the inner square root term:

$(a-x^2)^2 = a+x \implies a^2-2ax^2+(x^2)^2 = a+x \implies x^4-2ax^2-x+(a^2-a) = 0$

$\ldots$

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$

We started with the equation:

$\sqrt{a-\sqrt{a+x}} = x$

First, let's square both sides:

$a-\sqrt{a+x} = x^2 \implies \sqrt{a+x} = a-x^2$

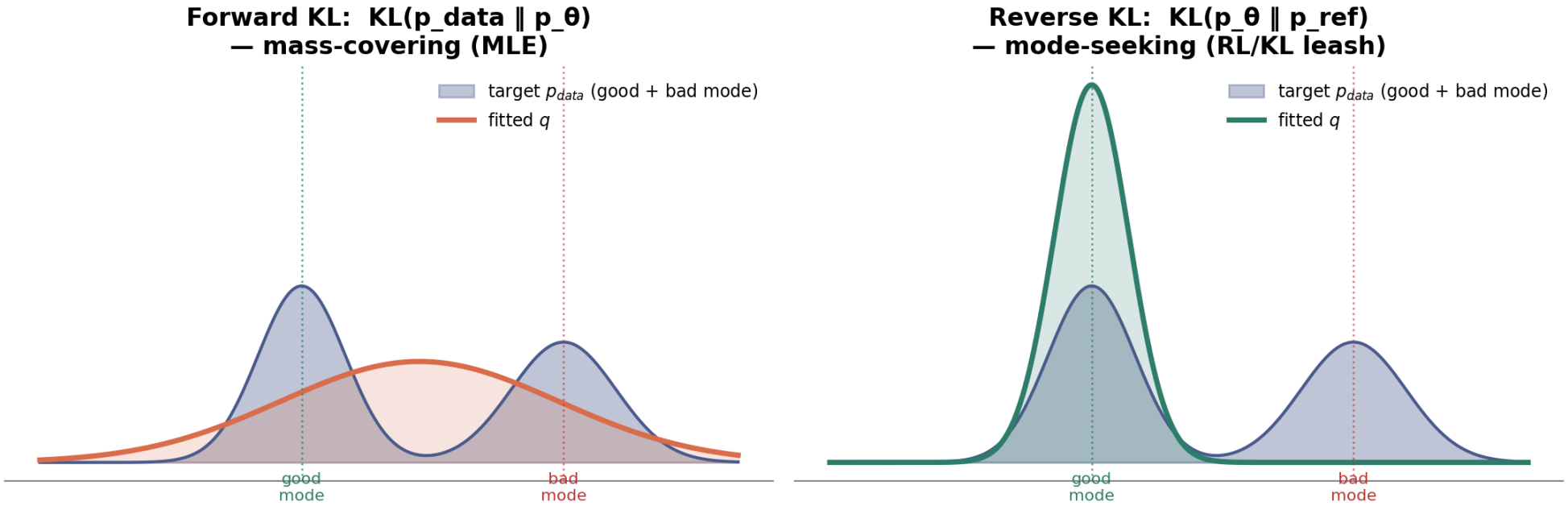
Next, I could square both sides again, treating the equation: $\cdots$

$\ldots$

Table 3 | An interesting “aha moment” of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

RLHF: Bradley Terry Model and the KL Leash

Why direction of KL matters: MLE spreads mass over every mode in the data (including bad ones);
the reference-KL penalty in RL lets the policy concentrate on one good mode instead



- **Preference loss (Bradley–Terry) trains a reward model on human comparisons; RL then optimizes reward minus a KL penalty against the reference policy:** the leash that keeps the optimizer from drifting into the reward model's blind spots.
- Forward KL (MLE) covers every mode in the data, including the bad one; reverse KL (the reference-KL leash used in RLHF) collapses onto a single good mode instead. Same bimodal target on both sides; only the fitting direction differs.

$$\arg\max_{\theta} \; \mathbb{E}_{x,y}\left[r(x,y)\right] - \beta\, D_{\mathrm{KL}}(p_\theta \,\|\, p_0)$$

Summary

	PPO	GRPO	DPO	RLHF	RLVR
What question does it answer?	How do we update the model?	How do we update the model?	How do we update the model?	Where does the reward come from?	Where does the reward come from?
In one line	Policy gradient, with a learned value network as the baseline	Same update, but the baseline is the average of K sampled answers	No RL loop: one supervised loss on "better vs worse" answer pairs	Reward is a model trained on human preference comparisons	Reward is a checker you can write: unit tests, exact match
Extra models to train	Value network (plus a reward model, if used with RLHF)	None; the group is its own baseline	None; just a frozen copy of the policy	A reward model	None; the checker is code
Main weakness	Expensive; the value network is unreliable early on	Wasted compute when a group is all-right or all-wrong	Off-policy: cannot correct the model's current mistakes	Length bias, sycophancy, gaming the reward model	Only works where a checker exists;
Typically seen in	InstructGPT-era chat alignment	DeepSeek-R1 and most reasoning models since	Open-weight chat models; preference fine-tuning APIs	Tone, helpfulness, safety	Math, code, anything checkable

Two different questions, often mixed up. ***PPO, GRPO and DPO are ways to update the model. RLHF and RLVR describe where the reward signal comes from.***
Every real system picks one of each: DeepSeek-R1 = GRPO + RLVR; classic ChatGPT alignment = PPO + RLHF.
The choice depends on the problem statement you want to solve.

Part 5: Current SOTA Models

Everyone Converged on the Same Recipe

	**OpenAI (*o-series, GPT*), xAI (Grok)**	RL on chain-of-thought; learns its own test-time strategy
	**Anthropic (Claude), Google (Gemini)**	RLHF-aligned base + reasoning / “thinking” modes
	**DeepSeek (R1, V-series)**	GRPO, largely reward-only, open weights
	**Alibaba (Qwen), Moonshot (Kimi), Zhipu (GLM)**	GRPO / RLVR-family pipelines, closing the gap with closed frontier

Focus on the descriptions— the pattern (RL on top of a strong pretrained prior) is what's converged; the exact recipe underneath is still moving month to month.

Leaderboard

Artifical Analysis's "Intelligence Index" scores a composite benchmark score by averaging a model's performance across reasoning, coding, and knowledge evals, roughly scaled 0-100. Higher means the model does better across that whole suite.

Open weights are catching up and China is leading that segment

- 4 out 7 leading models by Intelligence Index are all **Chinese and open-weight**.
- **Qwen** (Alibaba) alone **passed 1B+** cumulative **Hugging Face downloads** by March 2026 - over half of all open-source model downloads globally.
- Open weights matter operationally, not just philosophically: **you can fine-tune or RL-align the policy** on your own data, rules, and compliance constraints instead of renting someone else's alignment through an API.

Sources:
Artifical Analysis Aug 2026
Forbes, "China's DeepSeek V4 and Qwen Reshape the Open-Source AI Race" (Apr 28, 2026)

One Caveat, Stated Out Loud

- **Precise version numbers and benchmark scores in this space turn over in weeks**
- Focus on the shape of the landscape instead:
  - RLVR for anything checkable, RLHF (or a rubric hybrid) for anything that isn't, GRPO as the near-universal optimizer underneath both.
- **The algorithm commoditized faster than the models did,** which is exactly why Part 6 is about deployment and reward design, not “who has the best model.”

A good habit for reading any “state of AI” post after this course: ask whether the number is about the base model, the RL recipe, or the reward design — they age at very different speeds.

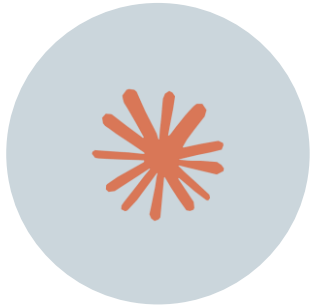
Part 6: Business Cases

Where It's Working: Coding Agents

Coding is close to an ideal RLVR domain: unit tests are a real, hard-to-game verifier

ANYSPHERE (CURSOR) $2B annualized revenue in March 2026, doubling in four months.

GitHub Copilot leads on raw seats (4.7M paid subscribers).



Claude Code leads on developer satisfaction.

COGNITION'S DEVIN represents the more autonomous end of the spectrum.

Where It's Working: In the industry

#	Company	Use Case	Agent Type	Verified ROI Metric
1	JPMorgan Chase	Investment Banking Automation	LLM Suite (in-house)	450+ AI use cases in production; $18B tech budget deployed
2	JPMorgan Chase	Contract Intelligence (COiN)	Document Review Agent (NLP)	360,000 lawyer-hours saved/year; 80% error reduction
3	Klarna	Customer Service Agent	Conversational AI (35+ languages)	$60M saved; 853 FTE equivalent; resolution time 11 min to under 2 min
4	Morgan Stanley	DevGen.AI: Code Modernization	GPT-based Code Review Agent	280,000 developer hours saved; 9M+ lines reviewed
5	Morgan Stanley	Wealth Management AI Assistant	CRM Sync and Notes Agent	98% voluntary advisor adoption rate
6	Salesforce	Legal Ops Contract Agent	Contract Drafting Agent	$5M+ in outside counsel costs eliminated
7	Walmart	Supply Chain Demand Forecasting	Autonomous Inventory Agent	4,700 stores connected to one forecasting system
8	PepsiCo / Siemens	Supply Chain Orchestration	Multi-step Orchestration Agent	CES 2026 announced capability; zero disruption in demo scenarios
9	Healthcare Providers	Clinical Documentation	Post-Consult Note Agent	42% reduction in documentation time per provider
10	General Mills	Supply Chain Optimization	Demand and Logistics Agent	$20M+ in savings since FY2024; 5,000+ daily shipments assessed
11	Singapore GovTech (VICA)	Citizen Services Platform	Hybrid NLP and Generative AI	800,000+ monthly queries resolved across 60+ agencies
12	JPMorgan Wealth Management	Personalized Outreach	Portfolio Comms Agent	20% increase in gross sales during market volatility periods

Where It's Working: Structured Knowledge Work

- **JPMorgan: agentic AI generating investment-banking pitch materials in ~30 seconds;** work that took analysts hours. 450+ AI use cases reportedly in production.
- Legal operations: contract review and redlining deployments reporting $5M+ in eliminated outside-counsel spend.
- **Shared trait: outputs are checkable against a template, a data source, or a policy;** verifiable-adjacent, even where not literally unit-tested.

A Reward-Proxy Cautionary Tale: Klarna

- **Feb 2024: AI assistant handled 2.3M chats in 30 days;** the work of 700 agents, automating 67% of conversations, projected $40M profit improvement.
- May 2025: Klarna reverses course and rehires humans. CEO: “we focused too much on efficiency and cost.”
- Customers reported generic answers on nuanced, emotionally loaded issues; some disputes raised compliance concerns.
- **“Customer satisfaction on a complex dispute” is exactly the learned-reward class from Part 4;** gameable the same way a reward model is.
- The 67%-automated metric looked like success and was measuring the wrong thing.

Part 7: Closing

Takeaways to Remember

RL does not add knowledge the base model lacked: *it reallocates probability mass toward what the reward prefers.*

Quick Summary

Part	Key Idea	Evidence
1. Why RL	MLE imitates; RL optimizes the real criterion directly	3 failure modes fixed
2–3. MDP & the Math	REINFORCE → baseline → PPO → GRPO → DPO	action space ≈ 10^5; GRPO = PPO − value net
4. Reward Regimes	RLVR (checkable) vs. RLHF (learned reward + KL leash)	2 regimes, 1 leash
5–6. Practice	RL pays off when the reward is chosen correctly	Cursor $2B ARR vs. Klarna's reversal

Thank you!